Q15: Create Shape abstract class with sub classes (Square, Circle and Rectangle) to show using dynamic Polymorphism.

```
package com.rani.basic;

abstract class Shape
{
  public abstract String area();
}

class Square extends Shape
{
  private int side;

  public Square(int side)
  {
    super();
    this.side = side;
  }

  @override
  public String area()
  {
    return "The area of Square is :"+side*side;
  }
}

class Circle extends Shape
{
  double radius;
  final double PI = 3.14;

  public Circle(double radius)
  {
    super();
    this.radius = radius;
  }

  @override
  public String area()
  {
    return "The area of Circle is :"+PI*radius*radius;
  }
}

class Rectangle extends Shape
{
  private int length, breadth;

  public Rectangle(int length, int breadth)
  {
    super();
    this.length = length;
    this.breadth = breadth;
  }

  @override
  public String area()
  {
    return "The area of rectangle is :"+length * breadth;
  }
}

public class ArrayExt9
{
  public static void main(String[] args)
  {
    Shape[] shape = new Shape[3];
    shape[0] = new Square(5);
    shape[1] = new Circle(10);
    shape[2] = new Rectangle(10, 20);

    getShape(shape);
  }

  public static void getShape(Shape [] shape)
  {
    for(Shape s : shape)
    {
      System.out.println(s.area());
    }
  }
}
```

Q21) Create sealed Animal interface and sub classes to show dynamic polymorphism.

```
package com.rani.basic;

sealed interface Animal
{
  void roam();
}
non-sealed class Bird implements Animal
{
  @override
  public void roam()
  {
    System.out.println("Bird is roaming");
  }
}

final class Lion implements Animal
{
  @override
  public void roam()
  {
    System.out.println("Lion is roaming");
  }
}

public class ArrayExt20
{
  public static void main(String[] args)
  {
    Animal [][animals] = new Animal[2];
    animals[0] = new Bird();
    animals[1] = new Lion();
    getAnimal(animals);
  }

  public static void getAnimal(Animal [] animals)
  {
    for(Animal animal : animals)
    {
      animal.roam();
    }
  }
}
```

Q22) WAP to sort an array elements.

```
package com.rani.basic;

import java.util.Arrays;

//WAP to sort an array elements
public class ArrayExt21
{
  public static void main(String[] args)
  {
    int [][arr = {5,3,2,8,7,10};

    System.out.println("Original Array :"+Arrays.toString(arr));

    int temp = 0;

    for(int i=0; i<arr.length; i++)
    {
      for(int j=i+1; j<arr.length; j++)
      {
        if(arr[i] > arr[j])
        {
          temp = arr[i];
          arr[i] = arr[j];
          arr[j] = temp;
        }
      }
    }

    System.out.println("Sorted Array :"+Arrays.toString(arr));
  }
}
```

Q23) Find the second largest element in the Array. (Stream API)

```
package com.rani.basic;

import java.util.Arrays;

//WAP to find the second largest number in an array

public class ArrayExt22
{
  public static void main(String[] args)
  {
    int [][arr = {5,3,2,8,7,10};

    System.out.println("Original Array :"+Arrays.toString(arr));

    int temp =0;

    for(int i=0; i<arr.length; i++)
    {
      for(int j=i+1; j<arr.length; j++)
      {
        if(arr[i] < arr[j])
        {
          temp = arr[i];
          arr[i] = arr[j];
          arr[j] = temp;
        }
      }
    }

    System.out.println("Second largest element is :"+arr[1]);
  }
}
```

Q23) Finding the duplicate elements in an Array.

```
package com.rani.basic;

//Finding the duplicate elements in an Array.
public class ArrayExt23
{
  public static void main(String[] args)
  {
    int [][arr = {2,4,5,7,5,2,4,2};

    for(int i=0; i<arr.length; i++)
    {
      for(int j=i+1; j<arr.length; j++)
      {
        if(arr[i] == arr[j])
        {
          System.out.println("Duplicate element is :"+arr[j]);
        }
      }
    }
  }
}
```

Q24) Insert a new element in the array at last position.

```
package com.rani.basic;

import java.util.Arrays;

//Insert a new element in the array at last position.

class InsertAtLast
{
  public static int[] insertAtLast(int []arr, int element)
  {
    int []newArray = new int[arr.length + 1];

    for(int i=0; i<arr.length; i++)
    {
      newArray[i] = arr[i];
    }

    newArray[newArray.length - 1] = element;

    return newArray;
  }

}

public class ArrayExt24
{
  public static void main(String[] args)
  {
    int [][arr = {10,20,30,40,50};
    System.out.println("Original Array :"+Arrays.toString(arr));

    int[] intAdd = InsertAtLast.insertAtLast(arr, 50);
    System.out.println("Array after insertion :"+Arrays.toString(intAdd));
  }
}
```

Q25) Insert a new element in the array at middle position.

```
package com.rani.basic;

import java.util.Arrays;

//Insert a new element in the array at middle position.

class InsertAtMiddle
{
  public static int[] insertAtMiddle(int []arr, int element)
  {
    int []newArray = new int[arr.length + 1];
    int middle = arr.length / 2;

    for(int i=0; i<middle; i++)
    {
      newArray[i] = arr[i];
    }

    newArray[middle] = element;

    for(int i=middle; i<arr.length; i++)
    {
      newArray[i+1] = arr[i];
    }

    return newArray;
  }
}
public class ArrayExt25
{
  public static void main(String[] args)
  {
    int arr[] = {10,20,30,40,50,60, 70};
    System.out.println("Original Array :"+Arrays.toString(arr));

    int[] intMiddle = InsertAtMiddle.insertAtMiddle(arr, 100);
    System.out.println("Array after insertion :"+Arrays.toString(intMiddle));
  }
}
```

Q26) Program to merge two array elements into a single Array.

```
import java.util.Arrays;
public class ArrayExt26
{
  public static int[] mergeArray(Elements[int [] x, int [] y)
  {
    int[] result = new int[x.length + y.length];
    int pos = 0;

    for (int element : x)
    {
      result[pos] = element;
      pos++;
    }
    for (int element : y)
    {
      result[pos] = element;
      pos++;
    }

    return result;
  }

  public static void main(String[] args)
  {
    int[] arrInt1 = {1, 2, 3, 4, 5};
    int[] arrInt2 = {11, 12, 13, 14, 15, 16};

    int[] result = mergeArray(Elements[arrInt1, arrInt2);

    System.out.println(Arrays.toString(result));
  }
}
```

Q27) Finding the unique element in the Array.

```
  import java.util.Arrays;
  import java.util.Scanner;

  public class ArrayExt27
  {
    public static void main(String[] args)
    {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter the size of the array :");
      int size = sc.nextInt();

      int []arr = new int[size];

      for(int i=0; i<arr.length; i++)
      {
        System.out.print("Enter the elements of the array :");
        arr[i] = sc.nextInt();
      }

      System.out.println("Original Array Elements are :"+Arrays.toString(arr));

      System.out.print("Unique elements in the array :");
      for (int i = 0; i < arr.length; i++)
      {
        boolean isUnique = true;
        for (int j = 0; j < i; j++)
        {
          if (i != j && arr[i] == arr[j])
          {
            isUnique = false;
            break;
          }
        }
        if (isUnique)
        {
          System.out.print(arr[i] + " ");
        }
      }

    }

  }
```